Mr.RC | 0xU 🅿️
@MrRyanChi
Wed Apr 01 10:00:01 +0000 2026

2026
如何量化交易

2026年，普通人如何量化交易

原文作者：@gemchange_ltd 原文链接：https://x.com/gemchange_ltd/status/2028904166895112617 编译、注释与改写：Mr.RC, insiders.bot (@insidersdotbot) 创始人，@0xUClub前会长 在2026年，量化交易是每个交易员的基础素养

原文作者：@gemchange_ltd

原文链接：https://x.com/gemchange_ltd/status/2028904166895112617

编译、注释与改写：Mr.RC, insiders.bot (@insidersdotbot) 创始人，@0xUClub前会长

在2026年，量化交易是每个交易员的基础素养

上个礼拜，我受到香港大学人工智能与管理协会（@camo_hku）的邀请，交流分享了Agent时代的那些搞钱方法。整个活动下来，我最大的收获就是一件事，那就是：

AI时代 = 技术平权的时代

过去，量化是小部分机构的专属。现在，无数工作室甚至个人参与到了创造量化策略，并且获得持续收益的过程当中。换言之，如果你还不理解量化的本质，那么你将会在市场上面临很大的劣势。

在OpenClaw盛行的今日，任何人都可以靠量化赚钱。但这需要两个前提。

> 第一，是基础设施，这正是我们在@insidersdotbot，通过制作Agent和算法原生的交易平台，数据库与Skills正在试图实现的。正式版基于Agent的回测功能，也会是这个生态的一部分。

> 第二，也是作为个体最重要的，就是架构能力和策略的设计能力。策略并不需要100%精确，但一定要独特，精巧，可以抓到别人意识不到的大机会。

只要你有专属于你的策略 + 牛逼的底层设施，那么，佐以Vibe Coding的赋能，你就离财富自由不远了。

而在学习策略与架构这件事情上，@gemchange_ltd 这篇原文，是我目前看到的、最完整的一份"量化交易知识地图"。它以预测市场为印子，把成为一名顶尖宽客（量化交易员/Quant）所需要的每一块拼图，按照正确的学习顺序，一次性讲清楚了。

相信看完之后，就算是小白，你也能了解如何开始量化交易，以及如何设计属于你的策略。

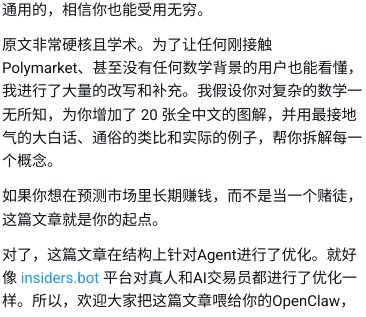
如果你是预测市场交易员，那这就是你必读的文章。

如果你是其他资产的交易者，这篇文章的很多思路都是通用的，相信你也能受用无穷。

原文非常硬核且学术。为了让任何刚接触Polymarket、甚至没有任何数学背景的用户也能看懂，我进行了大量的改写和补充。我假设你对复杂的数学一无所知，为你增加了 20 张中文的图解，并用最接地气的大白话、通俗的类比和实际的例子，帮你拆解每一个概念。

如果你想在预测市场里长期赚钱，而不是当一个赌徒，这篇文章就是你的起点。

对了，这篇文章在结构上针对Agent进行了优化。就好像 insiders.bot 平台对真人和AI交易员都进行了优化一样。所以，欢迎大家把这篇文章喂给你的OpenClaw，Manus，Claude，或者任何一个AI，然后立刻开始建立你的量化模型。



序言：你是在交易，还是在赌博？

先问你一个问题。

你在 Polymarket 上看到一个合约，"特朗普赢得大选"的 YES 价格是 $0.52。你觉得他赢的概率更高，于是花了 $520 买了 1000 股 YES。

你觉得你在做交易。但实际上，你只是在赌博。因为你没有回答过这些问题：

- 你的 52% 是怎么算出来的？
- 你的信息来源比市场上的其他参与者更好吗？
- 如果明天出了一条新闻，你的概率估计应该怎么更新？
- 你应该买多少仓位，才能在"万一猜错"的情况下不爆仓？

这些问题，不是靠"感觉"能回答的。它们需要数学。

2025 年，顶级量化公司（Jane Street、Citadel、HRT）的入门级宽客年薪在 $300K 到 $500K 之间。AI 和机器学习方向的金融招聘同比增长了 88%。这不是因为这些公司喜欢数学家。是因为数学真的能通过更正确的估值模型赚钱。

而 Polymarket，恰好是一个把所有量化金融核心概念完美融合在一起的交易市场：概率论、信息论、凸优化、整数规划，全都用得上。

第一章：概率，不确定性世界的唯一语言

大多数人对量化交易有一个巨大的误解。他们以为量化交易就是"选股"，是对某个事件有独到的见解。

其实根本不是。

量化交易的本质 = 纯数学。

而更具体的说，你在寻找的是：

- 统计学上的相关性
- 定价的低效
- 结构性的优势。

这些优势之所以存在，是因为市场是一个由人类组成的复杂系统，而人类总是会犯系统性的错误。

在量化金融的世界里，所有的问题最终都可以简化为一个问题：赔率是多少，以及这个赔率对我来说有多大的优势？

所以首先，你需要深刻理解"概率"的本质。

条件概率：告别绝对的对与错

普通人思考问题，喜欢用绝对的对与错。一件事要么发生，要么不发生。

> 但宽客的思考方式是条件式的。

> 他们会问：在已知某些信息的情况下，这件事发生的可能性有多大？

"已知某些信息时的概率"就是条件概率。

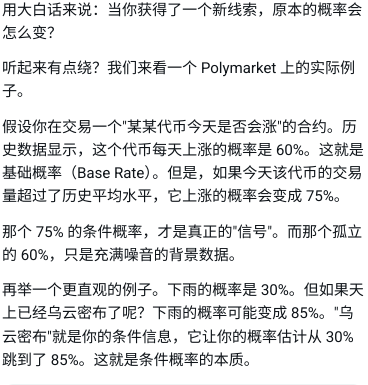
用大白话来说：当你获得了一个新线索，原本的概率会怎么变？

听起来有点晦？我们来看一个 Polymarket 上的实际例子。

假设你在交易一个"某某代币今天是否会涨"的合约。历史数据显示，这个代币每天上涨的概率是 60%。这就是基础概率（Base Rate）。但是，如果今天该代币的交易量超过了历史平均水平，它上涨的概率会变成 75%。

那个 75% 的条件概率，才是真正的"信号"。而那个孤立的 60%，只是充满噪音的背景数据。

再举一个更直观的例子。下雨的概率是 30%。但如果天上已经乌云密布了呢？下雨的概率可能变成 85%。"乌云密布"就是你的条件信息，它让你的概率估计从 30% 跳到了 85%。这就是条件概率的力量。



贝叶斯定理：如何实时更新你的信念

贝叶斯定理是量化交易的灵魂。它回答的问题是：当你获得了新的数据，你应该如何更新你原有的信念？

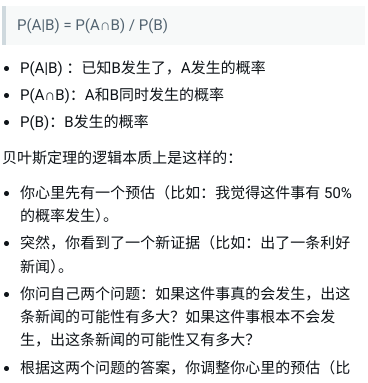
它的公式是这样的

> P(A|B) = P(A∩B) / P(B)

- P(A|B)：已知B发生了，A发生的概率
- P(A∩B)：A和B同时发生的概率
- P(B)：B发生的概率

贝叶斯定理的逻辑本质上是这样的：

- 你心里先有一个预估（比如：我觉得这件事有 50% 的概率发生）。
- 突然，你看到了一个新证据（比如：出了一条利好新闻）。
- 你问自己两个问题：如果这件事真的会发生，出这条新闻的可能性有多大？如果这件事根本不会发生，出这条新闻的可能性又有多大？
- 根据这两个问题的答案，你调整你心里的预估（比如：从 50% 调高到 58%）。


Heart of Bayes' theorem
All possibilities
All possibilities fitting the evidence
P(Libration given the evidence)

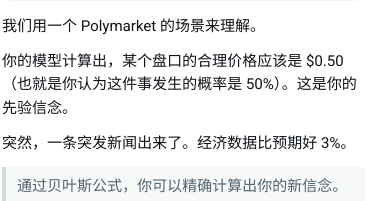
我们用一个 Polymarket 的场景来理解。

你的模型计算出，某个盘口的合理价格应该是 $0.50（也就是你认为这件事发生的概率是 50%）。这是你的先验信念。

突然，一条突发新闻出来了。经济数据比预期好了 3%。

> 通过贝叶斯公式，你可以精确计算出你的新信念。假设算出来是 58%。那你的新合理价格就是 $0.58。

在市场上，谁能最快、最准确地完成这种概率更新，谁就能赚最多的钱。这就是为什么一些量化团队要花几百万美元去建低延迟的系统。不是因为他们喜欢花钱，是因为快 0.1 秒就意味着多赚几万美元。



如果你想打好基础，去读一读哈佛大学免费的《Introduction to Probability》（概率论导论），前 6 章就够了。然后试着用 Python 写个代码，模拟抛 10,000 次硬币，亲眼看看大数定律是怎么运作的。

期望值与方差：你最好的两个朋友

在交易中，有两个数字比什么都重要。

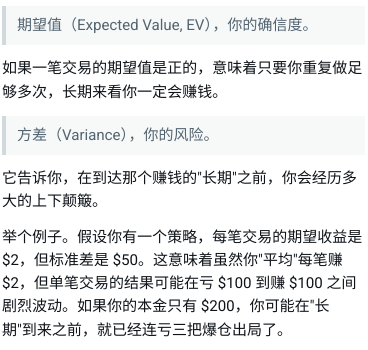
> 期望值（Expected Value, EV），你的确信度。

如果一笔交易的期望值是正的，意味着只要你重复做足够多次，长期来看你一定会赚钱。

> 方差（Variance），你的风险。

它告诉你，在到达那个"赚钱的"长期"之前，你会经历多大的上下颠簸。

举个例子。假设你有一个策略，每笔交易的期望收益是 $2，但标准差是 $50。这意味着虽然你"平均"每笔赚 $2，但单笔交易的结果可能在亏 $100 到赚 $100 之间剧烈波动。如果你的本金只有 $200，你可能在"长期"到来之前，就已经在短期亏损中破产了。



凯利公式：科学地决定下注大小

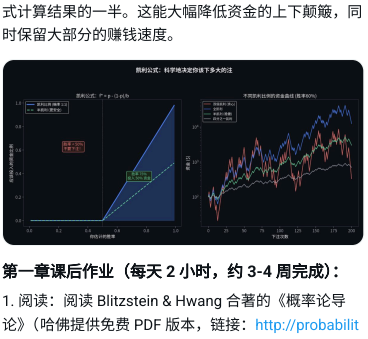
既然知道了期望值和方差，那面对一个好机会，我到底该买多少？全仓梭哈吗？

绝对不行。这里我们需要引入凯利公式（Kelly Criterion）。

> 凯利公式专门用来告诉你：在给定的胜率和赔率下，你应该把资金的百分之几押进去，才能让你的钱滚雪球滚得最快，同时又不会破产。

> 如果算出来是 20%，意味着你最多只能拿总资金的 20% 去下注。

在实战中，因为我们对胜率的估计往往有误差（你以为你有 60% 的胜率，其实可能只有 55%），顶尖的宽客通常会使用"半凯利"（Half Kelly），也就是只下注凯利公式计算结果的一半。这能大幅降低资金的回撤，同时保留大部分的赚钱速度。



第一章课后作业（每天 2 小时，约 3-4 周完成）：

1. 阅读：阅读 Blitzstein & Hwang 合著的《概率论导论》（哈佛提供免费 PDF 版本，链接：http://probabilitybook.net[1](https://stat110.hsites.harvard.edu/)）

2. 编程练习 1：模拟 10,000 次抛硬币，用图表直观验证"大数定律"。

3. 编程练习 2：实现一个贝叶斯更新器：输入先验概率和似然函数，输出后验概率。

```python
import numpy as np
import matplotlib.pyplot as plt

# 大数定律：随着实验次数增加，运行均值会逐渐趋近于真实概率
np.random.seed(42)
flips = np.random.choice([0, 1], size=10000, p=[0.5, 0.5])
running_avg = np.cumsum(flips) / np.arange(1, 10001)

plt.figure(figsize=(10, 4))
plt.plot(running_avg, linewidth=0.7)
plt.axhline(y=0.5, color='r', linestyle='--', label='真实概率')
plt.xlabel('抛硬币次数')
plt.ylabel('运行均值')
plt.title('大数定律的直观演示')
plt.legend()
plt.savefig('lln.png', dpi=150)
print(f"抛了 10,000 次后的均值: {running_avg[-1]:.4f} (真实值: 0.5000) ")
```

第二章：统计学 = 你的噪音探测器

当你学会了概率的语言，下一步就是学会"倾听数据"。

这就是统计学。

统计学教给我们的第一课就是：绝大多数看起来像"信号"的东西，其实都是噪音。

假设检验与多重比较陷阱

假设你写了一个交易机器人，回测数据显示它每年能赚 15%。这是真的吗，还是只是运气好？

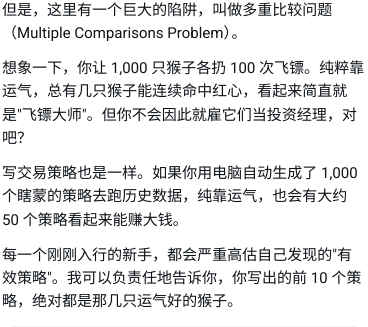
这时候你需要算一个 p 值（p-value）：如果这个策略其实是个垃圾（纯靠猜），它能碰巧跑出 15% 收益的概率有多大？统计学就是告诉你，这个概率有多少（比如小于 5%）。

但是，这里有一个巨大的陷阱，叫做多重比较问题（Multiple Comparisons Problem）。

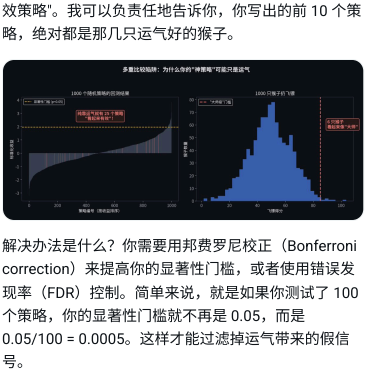
想象一下，你让 1,000 只猴子各自扔 100 次飞镖。纯粹靠运气，总有几只猴子能连续命中红心，看起来像是"飞镖大师"。但你不会因此就把它们当投资经理。对吧？

写交易策略也是一样。如果你用电脑自动生成了 1,000 个随意的策略去跑历史数据，纯靠运气，也会有大约 50 个策略看起来能赚大钱。

每一个刚刚入行的新手，都会严重高估自己发现的"有效策略"。我可以负责任地告诉你，你写出的前 10 个策略，绝对都是几只运气好的猴子。



解决办法是什么？你需要使用邦费罗尼校正（Bonferroni correction）来提高你的显著性门槛，或者使用错误发现率（FDR）控制。简单来说，就是如果你测试了 100 个策略，你的显著性门槛就不再是 0.05，而是 0.05/100 = 0.0005。这样才能过滤掉运气带来的假信号。



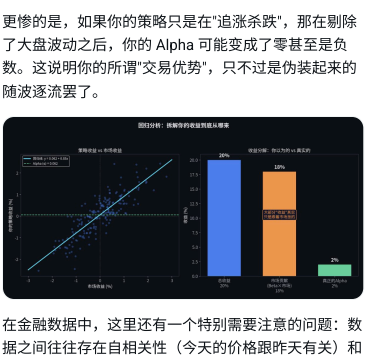
回归分析：拆解你的收益来源

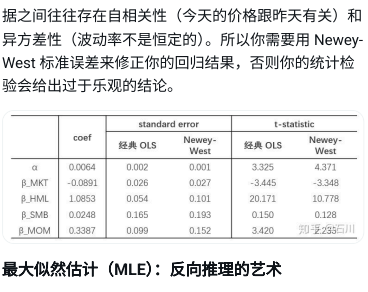
线性回归是金融界的主力工具。在量化交易中，你会把你的策略收益，跟大盘的涨跌放在一起对比。

> 这里的截距项 α（Alpha），就是你的超额收益。它是那些不能被大盘涨跌解释的、纯靠你个人技术赚到的钱。

举个例子。假设你的策略今年赚了 20%。但如果整个市场今年都翻倍涨了 18%，那你的技术得分（Alpha）其实只有 2%。

更惨的是，如果你的策略只是在"追涨杀跌"，那在剔除了大盘波动之后，你的 Alpha 可能变成了零甚至是负数。这说明你所谓的"交易优势"，只不过是伪装起来的随波逐流罢了。





	coef	standard error		t-statistic	
		经典 OLS	Newey-West	经典 OLS	Newey-West
α	0.0064	0.002	0.001	3.325	4.373
β_MKT	-0.0651	0.026	0.027	-3.445	-2.348
β_HML	1.0853	0.054	0.035	20.171	30.778
β_SMB	0.0045	0.025	[illegible]	0.150	[illegible]
β_MOM	0.0307	0.099	0.152	[illegible]	[illegible]

在金融数据中，这里还有一个特别需要注意的问题：数据之间往往存在自相关性（今天的价格跟昨天有关）和异方差性（波动率不是恒定的）。所以你需要用 Newey-West 标准误差来修正你的回归结果，否则你的统计检验会给出过于乐观的结果。

最大似然估计（MLE）：反向推理的艺术

当你听到一家顶级机构的宽客谈论他们在"校准"一个模型时，他们几乎永远在说一件事：最大似然估计（MLE）。

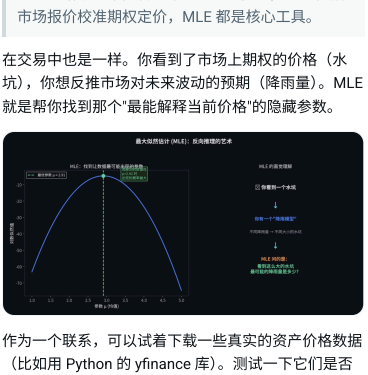
MLE 的原理其实很好理解，它就是一种"反向推理"。

打个比方。你在路边看到一个离地 2 米的水坑，你想知道昨晚下了多大的雨。你有一个"降雨模型"，告诉你不同降雨量会产生多大的水坑。

> MLE 做的事情就是反过来推：既然我已经看到了一个 2 米的水坑，那么在所有可能的降雨量中，哪一个降雨量最有可能造成这么大的水坑？

> 无论是波动率拟合还是 GARCH 模型，还是根据历史数据校准期权定价，MLE 都是核心工具。

在交易中也是一样。你看到了市场上期权的价格（水坑），你想反推市场对未来波动的预期（降雨量）。MLE 就是帮你找到那个"最能解释当前价格"的隐藏参数。



作为一个earlier，可以试着下载一些真实的资产价格数据（比如用 Python 的 yfinance 库）。测试一下它们是否符合正态分布。

剧透：绝对不符合。现实世界充满了肥尾效应（Fat Tails），也就是极端事件发生的频率远远高于正态分布的预期。试着用 MLE 去拟合一个 t-分布，看看真实的风险到底有多大。

第二章课后作业（约 4-5 周完成）：

1. 阅读：阅读 Wasserman 的《All of Statistics（统计学精要）》第 1 至第 13 章。（CMU 公开 PDF 版本：https://www.stat.cmu.edu/~brian/valerie/617-2022/0%20-%20books/2004%20-%20wasserman%20-%20all%20of%20statistics.pdf）

2. 编程练习 1：用 yfinance 下载真实的股票收益率数据，对其进行正态性检验（剧透：大概率会被拒绝，说明收益率并不服从正态分布）。然后用最大似然估计（MLE）拟合一个 t 分布，比较两者的差异。

3. 编程练习 2：使用 statsmodels 库，对一个股票组合跑 Fama-French 三因子回归。

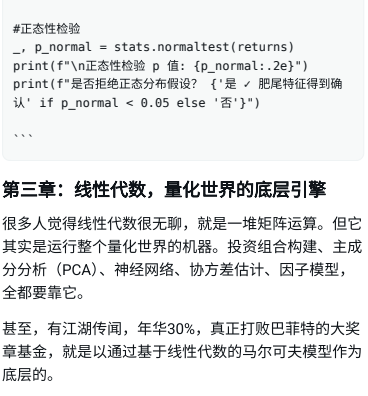
4. 编程练习 3：实现一个置换检验（Permutation Test）：将日期随机打乱 10,000 次，比较打乱后的表现与实际表现的差异。

```python
import numpy as np
from scipy import optimize, stats

# 演示"肥尾效应"：用最大似然估计 (MLE) 将 Student-t 分布拟合到收益率数据上

np.random.seed(42)
#模拟"接近真实"的收益率数据（带肥尾特征，略微正向漂移）
true_df = 4
returns = stats.t.rvs(df=true_df, loc=0.0005, scale=0.015, size=1000)

def neg_log_likelihood(params, data):
    df, loc, scale = params
    if df <= 2 or scale <= 0:
        return 1e10
    return -np.sum(stats.t.logpdf(data, df=df, loc=loc, scale=scale))

result = optimize.minimize(
    neg_log_likelihood, x0=[5, 0, 0.01], args=(returns,),
    method='Nelder-Mead'
)
fitted_df, fitted_loc, fitted_scale = result.x

print(f"MLE 拟合的自由度: {fitted_df:.2f} (真实值: {true_df})")
print(f"MLE 拟合的位置参数: {fitted_loc:.6f}")
print(f"MLE 拟合的尺度参数: {fitted_scale:.6f}")

#正态性检验
_, p_normal = stats.normaltest(returns)
print(f"\n正态性检验 p 值: {p_normal:.2e}")
print(f"是否拒绝正态分布假设？ {'是 - 肥尾特征得到确认' if p_normal < 0.05 else '否'}")
```

第三章：线性代数，量化世界的底层引擎

很多人觉得线性代数很无聊，就是一堆矩阵计算。但它其实是运行整个量化世界的机器。投资组合构建、主成分分析（PCA）、神经网络、协方差估计、因子模型，全都要靠它。

甚至，有江湖传闻，年华30%，真正打败巴菲特的大奖章基金，就是以线性代数为基础的马尔可夫模型作为底层的。



如果你不能流利地使用矩阵来思考，你就不可能成为一名宽客。

协方差矩阵：理解资产的联动

一个协方差矩阵 Σ（Sigma）捕捉了每一个资产相对于其他所有资产是如何移动的。

如果你在看 500 个币种，这个矩阵就是 500×500 的大小，包含 125,250 个独特的条目。每一个条目都在告诉你："当资产 A 涨的时候，资产 B 倾向于涨还是跌，涨跌的幅度有多大。"

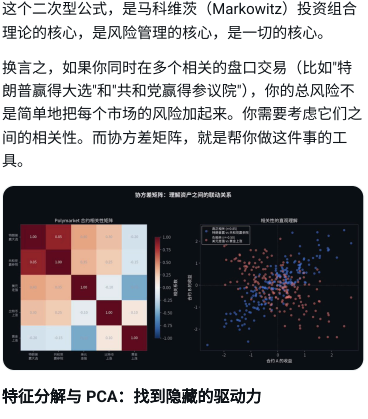
而整个投资组合的方差，可以明确成一个极其优雅的数学表达式：

> σ²_p = w^T Σ w

- w 是你的持仓权重向量
- Σ 是协方差矩阵

这个二次型公式，是马科维茨（Markowitz）投资组合理论的核心，是风险管理的核心，是一切的核心。

换言之，如果你同时在多个相关的盘口交易（比如"特朗普赢得大选"和"共和党赢得参议院"），你的总风险不是简单地把每个盘口的风险加起来。你需要考虑它们之间的相关性。而协方差矩阵，就是帮你做这件事的工具。



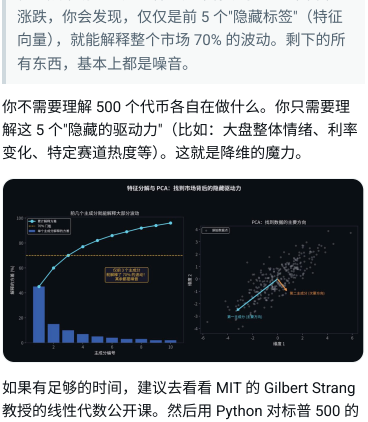
特征分解与 PCA：找到隐藏的驱动力

当你第一次使用特征分解（Eigendecomposition）进行主成分分析（PCA）时，你看待市场的方式都会改变。

主成分分析可以这样理解一个类比来解释：假设你要描述一个人的体型，你可以记录他的身高、体重、臂长、腿长、肩宽等几十个数据。但其实，这些数据很多都是联动的（个子高的人通常腿也长）。PCA 的作用，就是把这几十个数据的数据，浓缩成几个核心的"隐藏标签"，比如"整体块头大小"和"胖瘦程度"。

> 在金融市场里也是一样。如果你观察 500 个代币的涨跌，你会发现，仅仅是前 5 个"隐藏标签"（特征向量），就能解释整个市场 70% 的波动。剩下的所有东西，基本上都是噪音。

你不需要理解 500 个代币各自在做什么。你只需要理解 5 个"隐藏的驱动力"（比如：大盘整体情绪、利率变化、特定赛道热度等）。这就是降维的魔力。



如果有足够的时间，建议去看看 MIT 的 Gilbert Strang 教授的线性代数公开课。然后用 Python 对标普 500 的收益率做一次 PCA 分解，亲眼看看第一个主成分是什么。

你会发现，第一个主成分几乎等于"整个市场的涨跌"。

第三章课后作业（约 4-6 周完成）：

1. 观看视频：看完 Gilbert Strang 的 MIT 18.06 线性代数全部课程视频，一节都不能跳。（MIT OpenCourseWare 免费观看：https://ocw.mit.edu/courses/18-06-linear-algebra-spring-2010/video_galleries/video-lectures/）

2. 阅读：阅读 Strang 的《Introduction to Linear Algebra（线性代数导论）》，做书中的习题。（教材官网：https://math.mit.edu/~gs/linearalgebra/）

3. 编程练习 1：对标普 500 的收益率数据做 PCA（主成分分析）分解，画出特征值谱（即每个主成分解释了多少方差），找出前 3 个最重要的主成分。

4. 编程练习 2：从零开始实现 Markowitz 均值-方差优化。

```python
import numpy as np
import cvxpy as cp

# 使用 cvxpy 实现 Markowitz 投资组合优化

np.random.seed(42)
n_assets = 10
mu = np.random.uniform(0.04, 0.15, n_assets)
A = np.random.randn(n_assets, n_assets) * 0.1
cov = A @ A.T + np.eye(n_assets) * 0.01

w = cp.Variable(n_assets)
objective = cp.Minimize(cp.quad_form(w, cov))
constraints = [
    mu @ w >= 0.08, # 最低收益率要求
    cp.sum(w) == 1, # 满仓约束（权重之和为 1）
    w >= -0.1, # 最多允许 10% 的做空
    w <= 0.3 # 单一资产最多 30% 的仓位
]

prob = cp.Problem(objective, constraints)
prob.solve()

ret = mu @ w.value
vol = np.sqrt(w.value @ cov @ w.value)
sharpe = (ret - 0.03) / vol

print(f"组合预期收益率: {ret:.4f}")
print(f"组合波动率: {vol:.4f}")
print(f"夏普比率: {sharpe:.4f}")
print(f"各资产权重: {np.round(w.value, 4)}")
```

第四章：微积分与优化，捕捉变化的语言

微积分是关于"变化"的语言。在金融市场里，一切都在变：价格、波动率、相关性。而微积分就是帮你在以秒为单位变化条件。

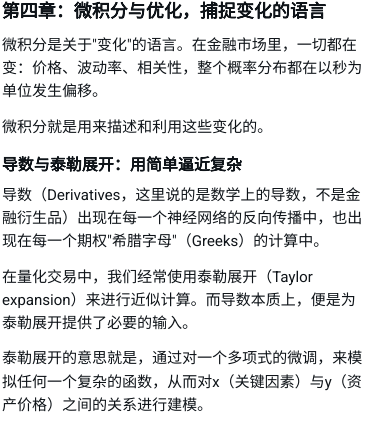
你需要用到微积分来描述和利用这些变化的。

导数与泰勒展开：用简单逼近复杂

导数（Derivatives，这里说的是数学上的导数，不是金融衍生品）告诉你在一个神经网络的反向传播中，也出现在每一个期权"希腊字母"（Greeks）的计算中。

在量化交易中，我们经常使用泰勒展开（Taylor expansion）来进行近似计算。而导数本质上，便是为泰勒展开提供了必要的输入。

泰勒展开的意思就是，通过对一个多项式的微调，来模拟任何一个复杂的函数。从而对x（关键因素）与y（资产价格）之间的关系进行建模。



假设你要画一条非常复杂的弯曲曲线，但你手里只有直尺。怎么办？

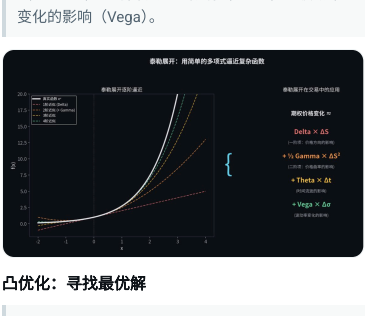
> 第一步，你用一根直线去贴合曲线的某一点（这叫一阶近似，在期权里叫 Delta）。在这一点附近，直线和曲线差不多。

> 第二步，如果你想贴合得更好，你可以把直线稍微掰弯一点，变成一个抛物线（这叫二阶近似，在期权里叫 Gamma）。

你掰弯的次数越多，你画出的线就越贴近那条复杂的曲线。

在交易中，期权的价格变化是一个极其复杂的公式。我们算不过来，所以就用泰勒展开，把它拆解成几个简单的部分：

> 价格方向的影响（Delta）+ 价格变动程度的影响（Gamma）+ 时间流逝的影响（Theta）+ 波动率变化的影响（Vega）。

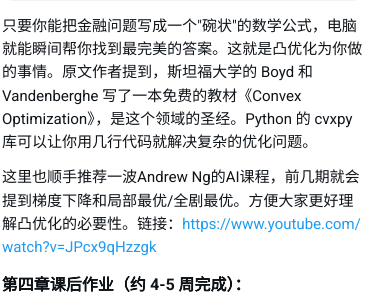


凸优化：寻找最优解

> 在量化金融中，几乎所有的"最优化"问题都可以表述为凸优化（Convex Optimization）问题。比如：在给定风险预算的情况下，怎么分配资金才能让收益最大化？

想象你被蒙上眼睛放在一个山谷里。要求你走到谷底（最优解）。

- 如果这个山谷坑坑洼洼，你可能会走到一个半山腰的坑里出不来（局部最优）。
- 但如果这是一个"碗"状的完美山谷，你只需要一直顺着下坡的方向走（梯度下降），你闭着眼睛都一定能走到最底部的那个唯一最低点（全局最优解）。



只要你能把金融问题写成一个"碗状"的数学公式，电脑就能瞬间帮你找到最完美的答案。这就是凸优化为你做的事情。原文作者建议，斯坦福大学的 Boyd 和 Vandenberghe 写了一本免费的教材《Convex Optimization》，是这个领域的圣经。Python 的 cvxpy 库可以让你用几行代码就解决复杂的优化问题。

这里也顺手推荐一波Andrew Ng的AI课程，前几周就会提到梯度下降和局部最优的问题。方便大家更好地理解优化的概念。链接：https://www.youtube.com/watch?v=JPcx9qHzzgk

第四章课后作业（约 4-5 周完成）：

1. 阅读：阅读 Boyd & Vandenberghe 的《Convex Optimization（凸优化）》第 1 至第 5 章。（斯坦福提供免费 PDF 版本：https://web.stanford.edu/~boyd/cvxbook/bv_cvxbook.pdf，书籍主页：https://stanford.edu/~boyd/cvxbook/）

2. 编程练习 1：从零开始实现梯度下降算法，用它来求 Rosenbrock 函数的最小值。（Rosenbrock 函数是优化领域最经典的测试函数之一，看起来简单但实际上很难优化，非常考验算法性能。）

3. 编程练习 2：用 cvxpy 求解一个投资组合优化问题，并加入交易成本约束。

第五章：随机微积分，从数据科学家到真正宽客的蜕变

在学会随机微积分之前，你只是一个喜欢金融的数据科学家。学会了它之后，你才是一个真正的宽客。

这是你学习如何在连续时间中对随机性进行建模的地方。在这里，你将从第一性原理推导出著名的 Black-Scholes 方程，并真正理解为什么衍生品的定价方式会是如今这种方式。

*注5.1：想要更好地了解 Black-Scholes 方程以及其含义，可以参考之前写的一篇"Polymarket做市圣经"。链接：https://x.com/MrRyanChi/status/2033466480067747844

*注5.2：为什么随机微积分（Itô calculus）和普通的微积分不一样？正是因为在随机微积分中，二阶泰勒项不会消失。在普通微积分里，当时间间隔趋于零时，二阶项可以忽略不计。但在随机微积分里，由于布朗运动的特殊性质，(dW)² = dt，二阶项变成了一阶的量级，不能忽略。详情见下文。

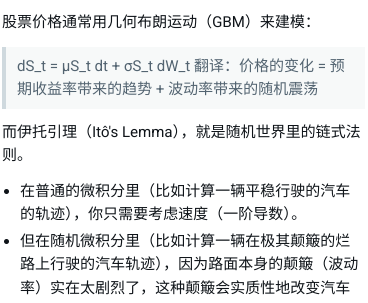
布朗运动：纯粹随机性的数学表达

布朗运动（也叫维纳过程，W_t）是一个连续时间的随机游走。

想象一个醉汉在广场上走路。他每一步的方向都是完全随机的。他走出的路线就是布朗运动。股票价格的跳动，在数学上就是被模拟成类似的醉汉随机游走。

布朗运动的例子还有很多，比如粒子扩散，空气粒子的运动也是随机的布朗运动。

这里有一个决定一切的性质：在布朗运动中，时间的流逝和距离的平方是等价的（即 (dW)² = dt）。正是因为这个性质，随机微积分才跟普通微积分不一样。



伊托引理：随机世界的链式法则

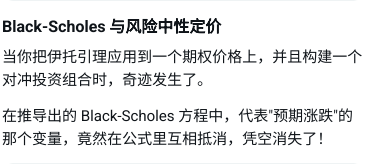
股票价格通常用几何布朗运动（GBM）来建模：

> dS_t = μS_t dt + σS_t dW_t 翻译：价格的变化 = 预期收益率带来的趋势 + 波动率带来的随机震荡

而伊托引理（Itô's Lemma），就是随机世界里的链式法则。

- 在普通的微积分里（比如计算一辆平稳行驶的汽车的轨迹），你只需要考虑速度（一阶导数）。
- 但在随机微积分里（比如计算一辆在极其颠簸的烂路上行驶的汽车轨迹），因为路面本身的颠簸（波动率）实在太剧烈了，这种颠簸会实质性地改变汽车的轨迹。

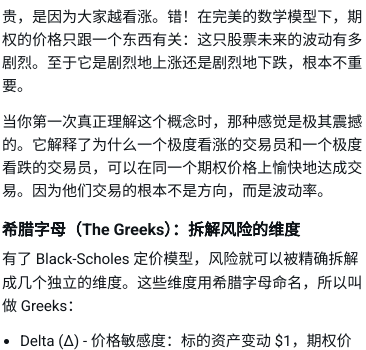
> 所以，伊托引理告诉我们：在计算随机变化时，你不能只看均方（一阶项），你必须把"颠簸程度"（二阶项）也加进公式里。如果不加，你算出来的价格就会错得离谱。



Black-Scholes 与风险中性定价

当你把伊托引理应用到一个期权价格上，并且构建一个对冲投资组合时，奇迹发生了。

在推导出的 Black-Scholes 方程中，代表"预期涨跌"的那个变量，竟然在公式里互相抵消，凭空消失了！



这意味着什么？这意味着期权的价格，根本不依赖于你对这只股票未来是涨是跌的预期。

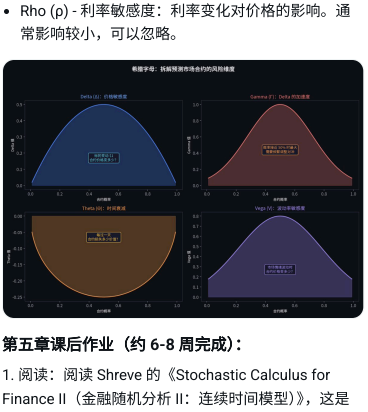
也就是说，假设你在买一个看涨期权。你以为期权越贵，是因为大家越看涨。错！在完美的数学模型下，期权的价格只跟一个东西有关：这只股票未来的波动有多剧烈。至于它是剧烈地上涨还是剧烈地下跌，根本不重要。

当你第一次真正理解这个概念时，那种感觉是极其震撼的。它解释了为什么一个极度看涨的交易员和一个极度看跌的交易员，可以在同一个期权价格上达成一致交易。因为他们交易的根本不是方向，而是波动率。

希腊字母（The Greeks）：拆解风险的维度

有了 Black-Scholes 定价模型，风险就可以被精确拆解成几个独立的维度。这些维度用希腊字母命名，所以叫 Greeks：

- Delta (Δ) - 价格敏感度：标的资产变动 $1，期权价格变动多少。它直接告诉你需要买多少现货来对冲风险。
- Gamma (Γ) - 弯曲程度：Delta 的变化速度。它告诉你需要多频繁地调整你的对冲仓位。在事件概率接近 50% 时，Gamma 最大，风险也最高。
- Theta (Θ) - 时间衰减：每过一天，期权损失多少价值。你可以把它理解为持有期权每天需要交的"租金"。
- Vega (V) - 波动率敏感度：波动率变化 1%，期权价格变多少。这是大多数华尔街衍生品交易台真正赚钱（或亏钱）的地方。
- Rho (ρ) - 利率敏感度：利率变化对价格的影响，通常影响较小，可以忽略。



第五章课后作业（约 6-8 周完成）：

1. 阅读：阅读 Shreve 的《Stochastic Calculus for Finance II（金融随机分析 II：连续时间模型）》。这是该领域公认的金标准教材。（PDF 版本：https://cms.dm.uba.ar/academico/materias/2docuat2016/analisis_cuantitativo_en_finanzas/Steve_ShreveStochastic_Calculus_for_Finance_II.pdf）

2. 备选教材：如果觉得 Shreve 太难啃，可以改读 Arguin 的《A First Course in Stochastic Calculus（随机微积分入门）》，这本书更新、更易读。（AMS 官方页面：https://bookstore.ams.org/amstext-53/）

3. 推导练习 1：对 f(S) = ln(S) 应用伊藤引理（Itô's Lemma），其中 S 服从几何布朗运动（GBM），推导出那个关键的 -σ²/2 修正项。（这个修正项是理解对数收益率和连续复利之间关系的核心。）

4. 推导练习 2：从 Delta 对冲论证出发，完整推导出 Black-Scholes 偏微分方程。

6. 编程练习：从零实现 Black-Scholes 公式定价，再用蒙特卡洛模拟定价，比较两者结果，验证蒙特卡洛模拟收敛到解析解的正确性。

```python
import numpy as np
from scipy.stats import norm

def black_scholes(S, K, T, r, sigma, option_type='call'):
    d1 = (np.log(S/K) + (r + sigma**2/2)*T) / (sigma*np.sqrt(T))
    d2 = d1 - sigma*np.sqrt(T)
    if option_type == 'call':
        return S*norm.cdf(d1) - K*np.exp(-r*T)*norm.cdf(d2)
    else:
        return K*np.exp(-r*T)*norm.cdf(-d2) - S*norm.cdf(-d1)

def monte_carlo_option(S0, K, T, r, sigma, n_sims=500_000):
    """通过风险中性模拟定价（注意漂移项用的是无风险利率 r，而不是 mu）"""
    Z = np.random.standard_normal(n_sims)
    ST = S0 * np.exp((r - sigma**2/2)*T + sigma*np.sqrt(T)*Z)
    payoffs = np.maximum(ST - K, 0)
    price = np.exp(-r*T) * np.mean(payoffs)
    stderr = np.exp(-r*T) * np.std(payoffs) / np.sqrt(n_sims)
    return price, stderr

def greeks(S, K, T, r, sigma):
    d1 = (np.log(S/K) + (r + sigma**2/2)*T) / (sigma*np.sqrt(T))
    d2 = d1 - sigma*np.sqrt(T)
    return {
        'delta': norm.cdf(d1),
        'gamma': norm.pdf(d1) / (S * sigma * np.sqrt(T)),
        'theta': -(S*norm.pdf(d1)*sigma)/(2*np.sqrt(T)) - r*K*np.exp(-r*T)*norm.cdf(d2),
        'vega': S * np.sqrt(T) * norm.pdf(d1),
        'rho': K * T * np.exp(-r*T) * norm.cdf(d2),
    }

# 验证：蒙特卡洛模拟的结果收敛于 Black-Scholes 解析解
S, K, T, r, sigma = 100, 105, 1.0, 0.05, 0.2

bs = black_scholes(S, K, T, r, sigma)
mc, err = monte_carlo_option(S, K, T, r, sigma)
g = greeks(S, K, T, r, sigma)

print(f"Black-Scholes 解析解: ${bs:.4f}")
print(f"蒙特卡洛模拟值: ${mc:.4f} ± {err:.4f}")
print(f"两者差异: ${abs(bs - mc):.4f}\n")
for name, val in g.items():
    print(f" {name:>6}: {val:.6f}")
```

第六章：Polymarket 与 LMSR，预测市场的数学引擎

现在，让我们把所有的数学武器，带回到当今世界上最有趣的交易市场：Polymarket。

Polymarket 背后的数学，完美地连接了这篇文章里提到的所有东西：概率论、信息论、凸优化和整数规划。

LMSR = 神经网络的 Softmax

在早期的预测市场中，自动做市商（AMM）通常使用一种叫做 LMSR（Logarithmic Market Scoring Rule）的机制。这是由经济学家 Robin Hanson 发明的。

它的成本函数是：

> C(q) = b · ln(Σ e^(q_i/b)) 其中：

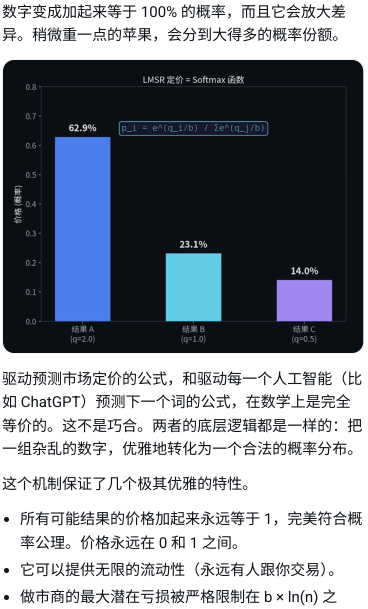
- q_i 是某个结果已发行的份额
- b 是流动性参数（b 越大，市场越"厚"，价格越难被大单推动）。

根据这个成本函数，我们可以算出某一个结果 i 对应的盘口价格：

> p_i = e^(q_i/b) / Σ_j e^(q_j/b)

如果你做一点机器学习，看到 LMSR 的价格计算公式，你会立刻觉得：这不就是 Softmax 函数吗！

什么是 Softmax？假设你有三个苹果，分别重 100 克、50 克、20 克。你想把它们的重量转换成"百分比概率"。Softmax 就是一个"概率转换器"。它不仅能把这些数字变成加起来等于 100% 的概率，而且它会放大差异，稍微重一点的苹果，会分到大得多的概率份额。



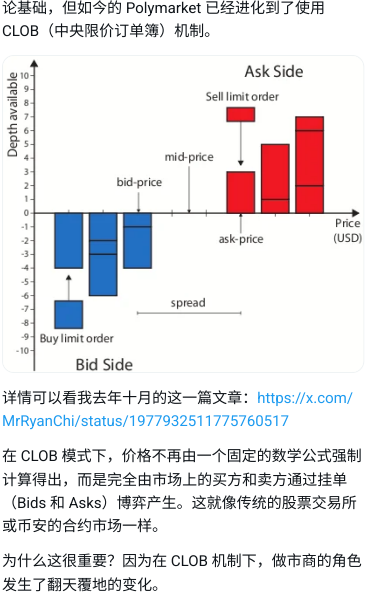
驱动预测市场定价的公式，和驱动每一个人工智能（比如 ChatGPT）预测下一个词的公式，在数学上是完全一样的。这不是巧合。两者的底层逻辑都是一样的：把一组杂乱的数字，优雅地转化为一个合法的概率分布。

这个机制保证了几个极其优雅的特性：

- 所有可能结果的价格加起来永远等于 1，完美符合概率公理。价格永远在 0 和 1 之间。
- 它可以提供无限的流动性（永远有人跟你交易）。
- 做市商的最大潜在亏损被严格限制在 b × ln(n) 之内。其中，n = 可能结果的数量。

Polymarket 的 CLOB 机制：从理论到实战

值得注意的是，虽然 LMSR 是预测市场 AMM 的经典理论基础。但如今的 Polymarket 已经进化到了使用 CLOB（中央限价订单簿）机制。



详情可以看我去年十月的这一篇文章：https://x.com/MrRyanChi/status/1977932511775760517

在 CLOB 模式下，价格不再由一个固定的数学公式强制计算得出，而是完全由市场上的买方和卖方通过挂单（Bids 和 Asks）博弈产生。这就像传统的股票交易所或币安的现货交易一样。

为什么这很重要？因为在 CLOB 机制下，做市商的角色发生了翻天覆地的变化。

LMSR (传统 AMM) 与 CLOB (Polymarket 当前) 的核心区别：

- 价格形成：LMSR 由数学公式自动计算；CLOB 由买卖双方挂单博弈产生。
- 流动性来源：LMSR 由系统资金池自动提供；CLOB 必须由做市商主动挂单提供。
- 做市商角色：在 LMSR 中不需要专业的做市商；但在 CLOB 中，做市商是市场的生命线。
- 价差控制：LMSR 的买卖价差由参数 b 决定；CLOB 的价差由做市商之间的竞争激烈程度决定。
- 对冲需求：在 CLOB 模式下，做市商面临极高的单边敞口风险，必须进行极其复杂的跨市场对冲。

用更简单的话说，在 LMSR 模式下，AMM 自动提供流动性，你只需要跟公式交易。但在 CLOB 模式下，流动性完全由做市商提供。你需要自己去计算合理的概率（使用前面提到的贝叶斯更新和概率模型），然后自己去挂买单和卖单，赚取买卖价差。

如果你在 Polymarket 上赚钱了做市，或者没有正确对冲相关联的风险，你挂出的单子就会被更聪明的量化资金瞬间吃掉，当成提款机。

第七章：宽客的职业版图与工具箱

如果你想把这套体系变成你的职业，或者组建自己的量化团队，你需要了解这个行业的生态。

四大核心角色

- 量化研究员（Quant Researcher）：在海量数据中寻找模式，构建预测模型的人。他们需要极高的数学和统计学天赋。在 Polymarket 的语境下，他们负责构建概率模型，判断一个合约的"合理价格"到底是多少。
- 量化开发工程师（Quant Developer）：构建基础设施的人。他们需要精通 C++、Rust 或 Python，打造低延迟的交易系统。在 Polymarket 的语境下，他们负责构建与 API 对接的交易引擎，确保订单能在毫秒级别内提交和执行。
- 量化交易员（Quant Trader）：管理资金、控制风险、做实时决策的人。他们的收入方差最大。在 Polymarket 上，他们就是那些在多个市场同时做市、实时调整价差和仓位的人。
- 风险宽客（Risk Quant）：团队的守护者。他们负责模型验证、计算极端情况下的最大亏损（VaR）和压力测试。

顶级机构的薪资水平

- 顶级公司（如 Jane Street, Citadel, HRT）：入门级新人年薪在 $300K 到 $500K 之间；资深员工年薪 $1M 到 $3M 以上；明星交易员可以拿到 $3M 到 $30M 以上。
- 中上游公司（如 Two Sigma, DE Shaw）：新人年薪 $250K 到 $350K；资深员工 $575K 到 $1.2M。

*注：Jane Street 在 2025 年上半年的平均员工薪酬高达 140 万美元。

推荐阅读清单（按学习顺序）

- 概率与统计 - Blitzstein & Hwang《Introduction to Probability》：条件概率、贝叶斯、分布
- 统计学进阶 - Wasserman《All of Statistics》：假设检验、回归、MLE
- 线性代数 - Strang《Introduction to Linear Algebra》：矩阵、特征值、PCA
- 优化理论 - Boyd & Vandenberghe《Convex Optimization》：凸优化理论与实践
- 随机微积分 - Shreve《Stochastic Calculus for Finance II》：布朗运动、伊托引理、BS 模型
- 量化金融 - Hull《Options, Futures, and Other Derivatives》：衍生品定价全景
- 实战策略 - Ernest Chan《Quantitative Trading》：从回测到实盘的完整流程

结语：我希望我早点知道的三个道理

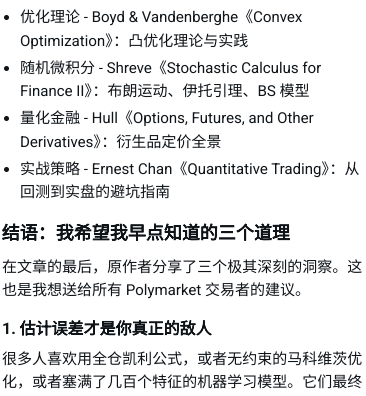
在文章的最后，原作者分享了三个极其深刻的洞察。这也是我想送给所有 Polymarket 交易者的建议。

1. 估计误差才是你真正的敌人

很多人喜欢用全仓去赌某个判断，或者无视概率的不确定性。但在实战中，凯利公式和马科维茨模型算出的优化结果，只是看起来很完美。它们对输入的参数极度敏感。如果你的概率估计差了几个百分点，整个优化结果就会完全错误。

数学在参数完美的情况下是完美的。但现实中，你永远得不到完美的参数。理论与实践之间的鸿沟，永远是估计误差。

最顶级的宽客，不是那些能搭建最复杂模型的人，而是那些对误差保持敬畏的人。他们会主动缩小仓位（用半凯利而不是全凯利），主动简化模型（用 3 个核心特征而不是 30 个），主动给自己留余地。



2. 工具已经民主化了，但"确信度"没有

今天，任何人都可以免费下载 PyTorch。任何人都可以接入 Polymarket 的 API。技术是必要条件，但不再是充分条件。

真正的交易优势（Edge），存在于独特的数据、独特的模型，或者独特的执行能力中。不是比别人多装了几个 Python 库。

这也是为什么我们把 @insidersdotbot 的更新推迟了整整一个月，来优先完善我们的聪明钱榜单，以及更好的PNL计算算法（比如比官方更准确的Split收益计算模式）。因为独特的数据和模型，真的能帮你赚更多的钱，或者转亏为盈。

在 Polymarket 上，这意味着什么？

意味着你需要找到别人没有的信息来源（比如某个小众领域的专家网络），或者构建别人没有的模型（比如一个能实时处理多市场相关性的定价引擎），或者拥有别人没有的执行能力（比如一个能在 10 毫秒内完成跨市场对冲的交易系统）。

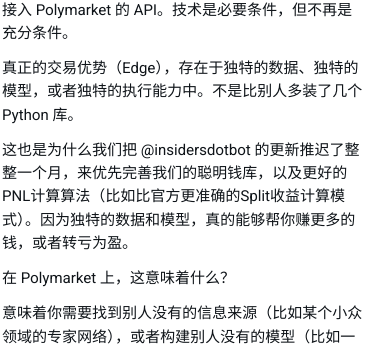
3. 数学才是真正的护城河

AI 可以帮你写代码，甚至可以建议交易策略。但是，能够推导出为什么伊托引理多了一项，能够证明在风险中性测度下折现价格是鞅（Martingale），能够判断在一个组合市场中凸松弛（Convex relaxation）何时是紧的。

这种深厚的数学直觉，才是区分"创造优势的宽客"和"借用优势的宽客"的根本分水岭。而借来的优势，迟早是会过期的。

预测市场正在经历传统期权市场在 1973 年经历过的变革。那些能够率先把严谨的数学模型、波动率定价和复杂的套利算法引入这个市场的人，将会拿走最大的红利。

停止靠直觉下注吧。去学概率，去写代码，去构建你的数学护城河。



完整工具箱

Python 技术栈

数据处理：pandas、polars（Polars 在处理大数据集时比 pandas 快 10 到 50 倍）

数值计算：numpy、scipy

机器学习（表格数据方向）：xgboost、lightgbm、catboost

机器学习（深度学习方向）：pytorch

优化求解：cvxpy

衍生品定价：QuantLib（工业级别的库，底层是 C++ 写的，性能很强）

统计分析：statsmodels

回测框架：NautilusTrader

回测框架（更简单易上手的选择）：backtrader、vectorbt（适合入门）

量化研究平台：Microsoft Qlib（GitHub 上超过 17000 颗星，偏 AI 方向）

强化学习交易：FinRL（GitHub 上超过 10000 颗星）

C++ 和 Rust

C++ 常用库：QuantLib、Eigen、Boost

Rust 方面：RustQuant 可以用来做期权定价，NautilusTrader 采用的是 Rust + Python 混合架构（底层核心用 Rust 保证速度，上层用 Python API 方便做研究）。

数据源

免费的：yfinance、Finnhub（每分钟 60 次请求限制）、Alpha Vantage

中等价位的：Polygon.io（每月 199 美元，延迟低于 20 毫秒）、Tiingo

企业级的：Bloomberg Terminal（彭博终端，大约每年 32000 美元）、Refinitiv、FactSet

区块链数据：Alchemy（有免费套餐，支持历史存档数据访问）

除此之外，@insidersdotbot 即将开源API。将会包含现成的聪明钱数据库与交易功能。欢迎点击小铃铛Stay Tuned。

求解器

Gurobi：速度最快的商用混合整数规划求解器，学生和学术用户可以申请免费许可证。做组合套利类问题离不开它。

Google OR-Tools：免费求解器里面最强的。

PuLP / Pyomo：Python 建模接口，用来方便地定义和调用各种求解器。

参考文献

[1] gemchanger. (2025). How I'd Become a Quant If I Had to Start Over Tomorrow. X. https://x.com/gemchange_ltd/status/2028904166895112617

[2] Blitzstein, J. K., & Hwang, J. (2014). Introduction to Probability. CRC Press. https://projects.iq.harvard.edu/stat110

[3] Markowitz, H. (1952). Portfolio Selection. The Journal of Finance.

[4] Strang, G. MIT 18.06 Linear Algebra. MIT OpenCourseWare. https://ocw.mit.edu/courses/18-06-linear-algebra-spring-2010/

[5] Boyd, S. & Vandenberghe, L. (2004). Convex Optimization. Cambridge University Press. https://web.stanford.edu/~boyd/cvxbook/

[6] Hanson, R. (2003). Logarithmic Market Scoring Rules for Modular Combinatorial Information Aggregation.

[7] Polymarket Documentation. CLOB Overview & API. https://docs.polymarket.com/trading/overview

[8] Black, F., & Scholes, M. (1973). The Pricing of Options and Corporate Liabilities. Journal of Political Economy.

[9] Shreve, S. (2004). Stochastic Calculus for Finance II: Continuous-Time Models. Springer.

Source / Copyright attribution
Author: @MrRyanChi (Mr.RC | 0xU 🅿️)
Post: https://x.com/mrryanchi/status/2039281623418695791
Fetched: 2026-09-28
Images and text copyright remain with the original author / X (Twitter).
Archived for personal reference; do not redistribute without permission.